

Dendê Eventos: Persistência e Exceções

Autor: Anna Beatriz Silva Lima
Fátima Pereira Santos Pinho
Ian Salomão da Silva Carneiro
Rebeca Helen Batista Amorim
Valeria Soares Santos

Agenda

Apresentar a evolução da solução
Dendê Eventos com persistência
real em banco de dados via JDBC
e tratamento de exceções
customizadas.

1. Contexto da Refatoração

Transição de Memória para
MySQL

2. Diagrama de Sequência

Fluxo da User Story Alterar
Evento

3. JDBC e Conexões

ConfigProperties e
ConnectionPool

4. CrudRepository e Generics

Padronização e Reuso

5. Construção dos Repositories

Implementação de operações
CRUD

6. RowMapper: criando objetos do modelo

Mapeamento e conversão de
ResultSet do JDBC para
entidades de domínio

7.Exceções Customizadas

Criação de mensagens de erro
personalizadas

Contexto da Refatoração



A solução

API Java da OAT2 com controllers, mappers, DTOs, builder e armazenamento em memória (HashMap).

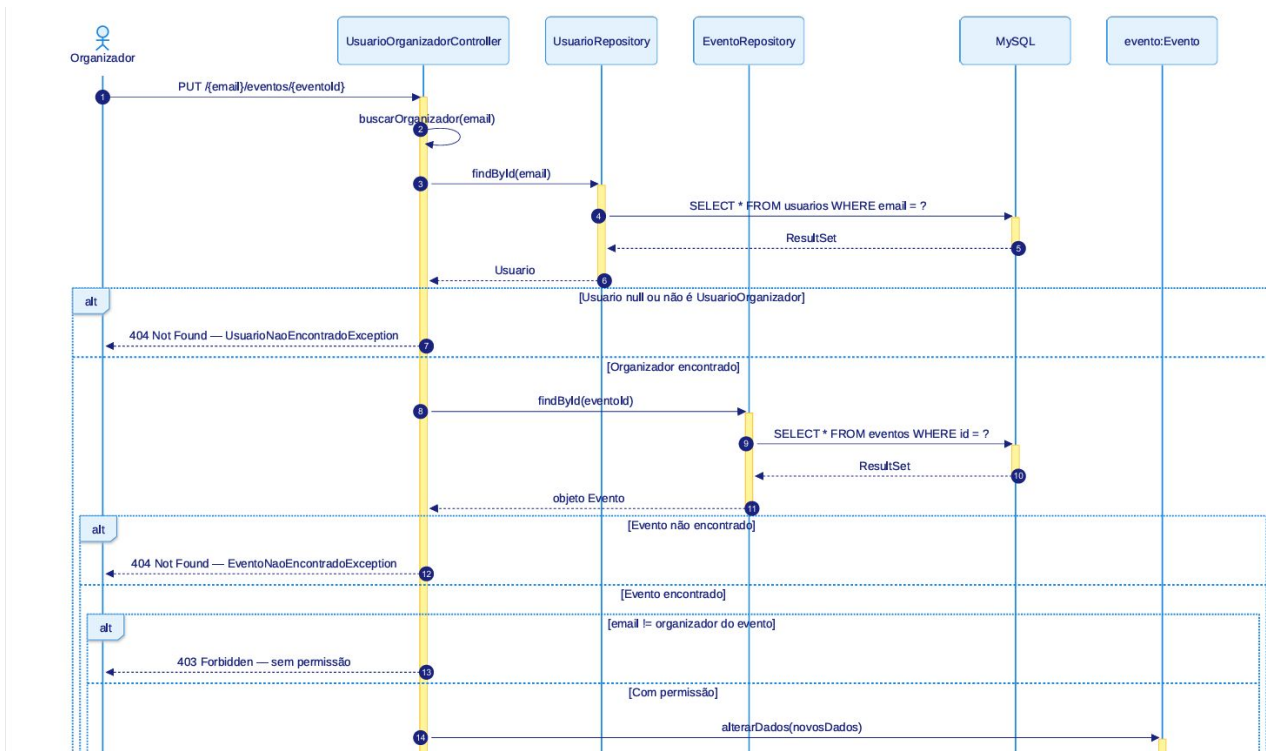
O desafio

Substituir o Repositório em memória por persistência real no MySQL usando apenas JDBC e tratar exceções conhecidas.

O complemento

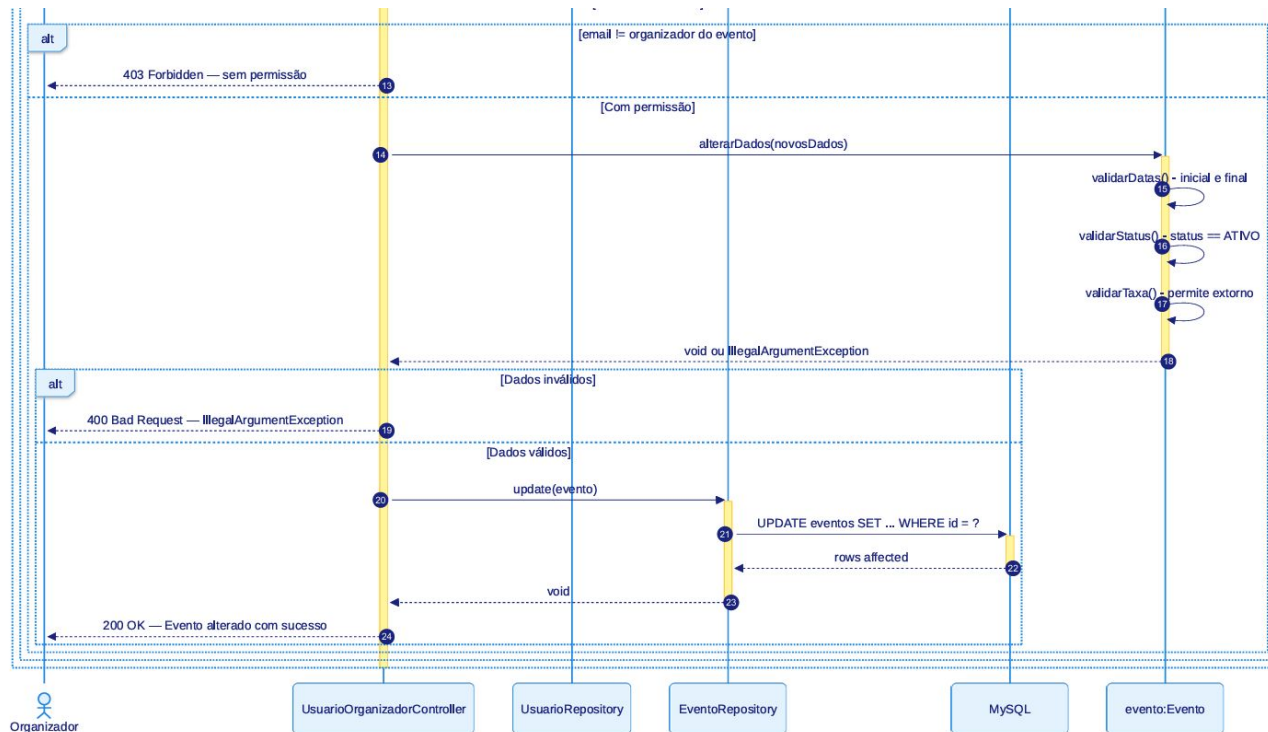
CrudRepository, RowMapper, ConnectionPool, ConfigProperties, exceções customizadas e Diagrama de Sequência.

Diagrama de Sequência (parte 1/2)



Dendê Eventos: Persistência e Exceções

Diagrama de Sequência (parte 2/2)



JDBC e Conexões:

ConfigProperties

- As configurações do banco ficam no **application.properties**
- A annotation **@Value** do **dendeframework** injeta cada propriedade automaticamente na classe
- Sem hardcoded de credenciais no código

```
1 public class ConfigProperties {
2
3     @Value(key = "datasource.url")
4     private String url;
5
6     @Value(key = "datasource.username")
7     private String username;
8
9     @Value(key = "datasource.password")
10    private String password;
11
12    @Value(key = "datasource.hikari.maximum-pool-size")
13    private int maximumPoolSize;
14 }
```

JDBC e Conexões:

ConnectionPool

- **ConnectionPool** é um Singleton o que garante uma única instância do pool em toda a aplicação
- Usa **HikariConfig** + **HikariDataSource** para gerenciar conexões com o banco
- O **double-checked** locking torna a criação thread-safe

```
1 public class ConnectionPool {
2
3     private static ConnectionPool instance;
4     private final HikariDataSource dataSource;
5
6     private ConnectionPool() {
7         ConfigProperties props = new ConfigProperties();
8         HikariConfig config = new HikariConfig();
9         config.setJdbcUrl(props.getUrl());
10        config.setUsername(props.getUsername());
11        config.setMaximumPoolSize(props.getMaximumPoolSize());
12        this.dataSource = new HikariDataSource(config);
13    }
14
15    public static ConnectionPool getInstance() {
16        if (instance == null) {
17            synchronized (ConnectionPool.class) {
18                if (instance == null) instance = new ConnectionPool();
19            }
20        }
21        return instance;
22    }
23
24    public Connection getConnection() throws SQLException {
25        return dataSource.getConnection();
26    }
27 }
```

Generics e CrudRepository



Interface Genérica

Padronização do acesso aos dados utilizando **Generics<T, ID>**

Permite que diferentes entidades (Evento, Usuário, Ingresso) utilizem a mesma estrutura de contrato para CRUD



```
public interface CrudRepository<T, ID> {  
    void save(T entity);  
    T findById(ID id);  
    List<T> findAll();  
    void update(T entity);  
    void delete(ID id);  
}
```


Construção dos Repositories:

save e findById



save

```
1 public void save(Evento evento) {
2     String sql = ""
3         INSERT INTO eventos (nome, descricao, ...)
4         VALUES (?, ?, ...)
5         "";
6
7     try (Connection conn = ConnectionPool.getInstance().getConnection();
8         PreparedStatement stmt = conn.prepareStatement(sql,
9             Statement.RETURN_GENERATED_KEYS)) {
10
11         preencherStatement(stmt, evento);
12         stmt.executeUpdate();
13
14         try (ResultSet keys = stmt.getGeneratedKeys()) {
15             if (keys.next()) evento.atribuirId(keys.getLong(1));
16         }
17     } catch (SQLException e) {
18         throw new OperacaoNaoPermitidaException(e.getMessage());
19     }
20 }
```

findById

```
1 @Override
2 public Evento findById(Long id) {
3     String sql = "SELECT * FROM eventos WHERE id = ?";
4
5     try (Connection conn = ConnectionPool.getInstance().getConnection();
6         PreparedStatement stmt = conn.prepareStatement(sql)) {
7
8         stmt.setLong(1, id);
9         try (ResultSet rs = stmt.executeQuery()) {
10             if (rs.next()) return new EventoRowMapper().mapRow(rs);
11         }
12     } catch (SQLException e) {
13         throw new OperacaoNaoPermitidaException(e.getMessage());
14     }
15     throw new EventoNaoEncontradoException(id);
16 }
```

Construção dos Repositories: update



Destaque: O método **preencherStatement()** centraliza a atribuição dos parâmetros evitando repetição entre **save** e **update**.

update

```
1  @Override
2  public void update(Evento evento) {
3      String sql = ""
4          UPDATE eventos SET
5              nome = ?, descricao = ?, data_inicio = ?,
6              data_final = ?, tipo = ?, modalidade = ?,
7              status = ?, preco_ingresso = ?, ...
8          WHERE id = ?
9      "";
10
11      try (Connection conn = ConnectionPool.getInstance().getConnection();
12          PreparedStatement stmt = conn.prepareStatement(sql)) {
13
14          preencherStatement(stmt, evento);
15          stmt.setLong(16, evento.getId());
16          stmt.executeUpdate();
17
18      } catch (SQLException e) {
19          throw new OperacaoNaoPermitidaException(e.getMessage());
20      }
21  }
```

RowMapper: criando objetos do modelo



- **RowMapper<T>** converte uma linha do **ResultSet** em um objeto de domínio
- Implementado como classe interna privada dentro de cada **RepositoryImpl**
- Encapsula toda a leitura de colunas do banco e a construção do objeto



```
1 public interface RowMapper<T> {  
2     T mapRow(ResultSet rs) throws  
   SQLException;  
3 }
```

RowMapper:

EventoRowMapper



```
1 private static class EventoRowMapper implements RowMapper<Evento> {
2
3     @Override
4     public Evento mapRow(ResultSet rs) throws SQLException {
5         LocalDateTime dataInicio = rs.getTimestamp("data_inicio")
6             .toLocalDateTime();
7         TipoEvento tipo = TipoEvento.valueOf(rs.getString("tipo"));
8         StatusEvento status = StatusEvento.valueOf(rs.getString("status"));
9
10        Boolean permiteEstorno = rs.getBoolean("permite_estorno");
11        if (rs.wasNull()) permiteEstorno = null;
12
13        Evento evento = new Evento(
14            rs.getString("nome"), rs.getString("descricao"),
15            dataInicio, tipo, status, ...
16        );
17        evento.atribuirId(rs.getLong("id"));
18        evento.restaurarStatus(status); // restaura sem acionar regras de negócio
19        return evento;
20    }
21 }
```

EventoRowMapper – lê o
ResultSet e monta o objeto
Evento

Exceções Customizadas



- Todas as exceções são **RuntimeException** (UncheckedException)
- Lançadas nos repositórios quando uma entidade não é encontrada ou uma operação falha
- O controller captura e retorna o status HTTP correto

exceptions

-  CapacidadeExcedidaException
-  DadosInvalidosException
-  EmailJaCadastradoException
-  EventoNaoEncontradoException
-  IngressoNaoEncontradoException
-  OperacaoNaoPermitidaException
-  UsuarioNaoEncontradoException

Considerações finais



- Persistência real em banco MySQL via JDBC puro (sem ORM)
- `CrudRepository<T, ID>` padroniza o acesso a dados com Generics
- `RowMapper<T>` isola a criação dos objetos do modelo, sem vaziar SQL para o domínio
- `ConnectionPool` com HikariCP gerencia conexões de forma eficiente
- Exceções customizadas cobrem todos os cenários conhecidos

Referências



- ORACLE. **Java Platform, Standard Edition 17: API Specification.** [Redwood Shores]: Oracle, 2021.
Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/index.html>. Acesso em: 20 fev. 2026.